



Lecture 16

Docker

Sharif University of Technology
Winter 2026



Learning Objectives

Why Containers? The Problem Space

Docker Core – Images, Containers, CLI

Networking, Volumes & Best Practices

Docker Compose – Multi-Service Apps

Orchestration – Swarm Mode Overview



Part I

Why Containers?

The problem Docker was built to solve

"It Works on My Machine!"



Sound familiar? This is one of software's oldest pain points.



Dev Laptop

Python 3.9, Node 16,
MySQL 8.0



Test Server

Python 3.8, Node 14,
MySQL 5.7



Production

Python 3.11, Node 18,
MySQL 8.4



Different environments = unpredictable bugs & deployment failures

Traditional Deployment Challenges



Dependency Hell

App A needs libssl 1.0, App B needs 1.1 — can't coexist on same host



Slow Provisioning

Spinning up a new server takes hours to days with manual config



Snowflake Servers

Every server is unique, hand-crafted — impossible to reproduce exactly



Wasted Resources

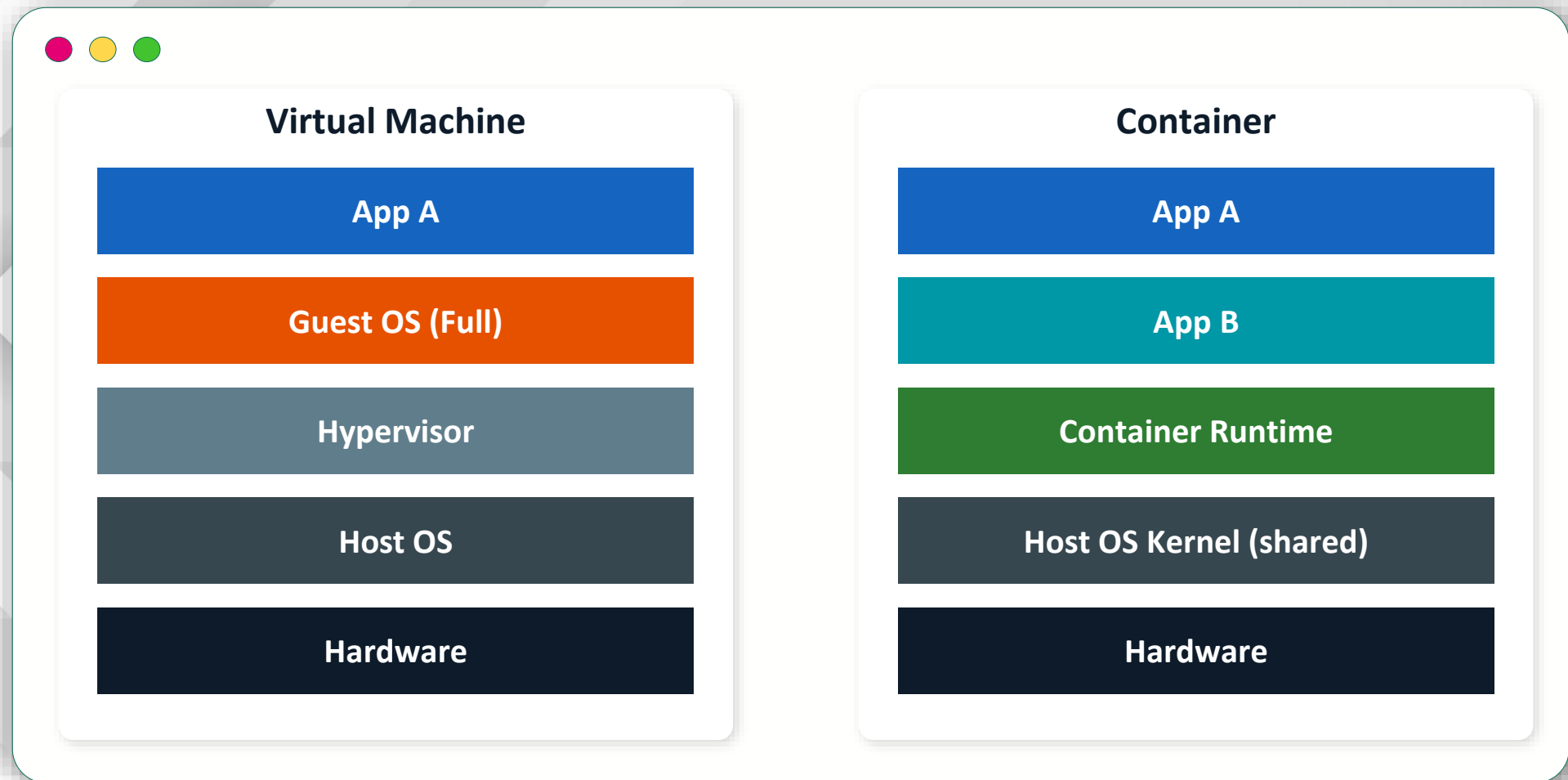
One app per VM = 80% idle CPU & RAM most of the time



Dev/Prod Parity

"Works in dev" doesn't mean "works in prod" — different Oses, paths, libs

VMs vs Containers — The Key Difference



Containers Are NOT Magic



A container is just a process on your Linux machine, with its view of the world restricted using kernel features.

namespaces



Isolation

Each container gets its own restricted view of the system — PID, network, filesystem, users.

cgroups



Resource Limits

Control Groups limit how much CPU, RAM, and disk I/O a container can consume.

UnionFS



Layered Filesystem

Images are built as stacked layers. Only changed layers are stored — saving disk space.

Namespaces — Isolation Without a VM

The Hotel Room Analogy

Imagine a large hotel building.

Hotel = Host Machine
Each Room = One Container

From inside your room you see only your TV, your wifi, your bathroom. You have no idea what's in room 204. The building's plumbing (kernel) is shared — but each room is isolated.



PID Namespace

Container's process tree starts at PID 1. Can't see host or sibling processes.



NET Namespace

Own virtual network interface, routing table, and ports — eth0 is the container's alone.



MNT Namespace

Own mount table. The container thinks it owns the entire filesystem.



USER Namespace

Root inside container $\neq$ root on host. Safer privilege model.

cgroups — Resource Limits & Accounting

The Shared Office Analogy

Imagine a co-working office.

Without rules → one loud tenant can blast music, hog all power outlets, and fill the fridge.

With cgroups → each tenant gets exactly: 2 power outlets, 40% of AC, 10 GB locker. They cannot take more, no matter how hard they try.



CPU

e.g. `--cpus="0.5"`

Container gets at most 50% of one CPU core. Scheduler enforces this hard.



Memory

e.g. `--memory="256m"`

If the container exceeds 256 MB, the OOM killer terminates its process — not the host.



Disk I/O

e.g. `--device-write-bps`

Limits MB/s a container can write to block devices, preventing I/O starvation for others.

UnionFS — The Layered Filesystem



Image Stack (OverlayFS)

Container R/W Layer

(your changes — ephemeral)

App Layer

node_modules, app code

Runtime Layer

Node.js 20 binary

Base OS Layer

Ubuntu 22.04 slim



Shared Base Layers

100 containers using the same Ubuntu base share ONE copy on disk. Only their diff layers are unique.



Like Git Commits

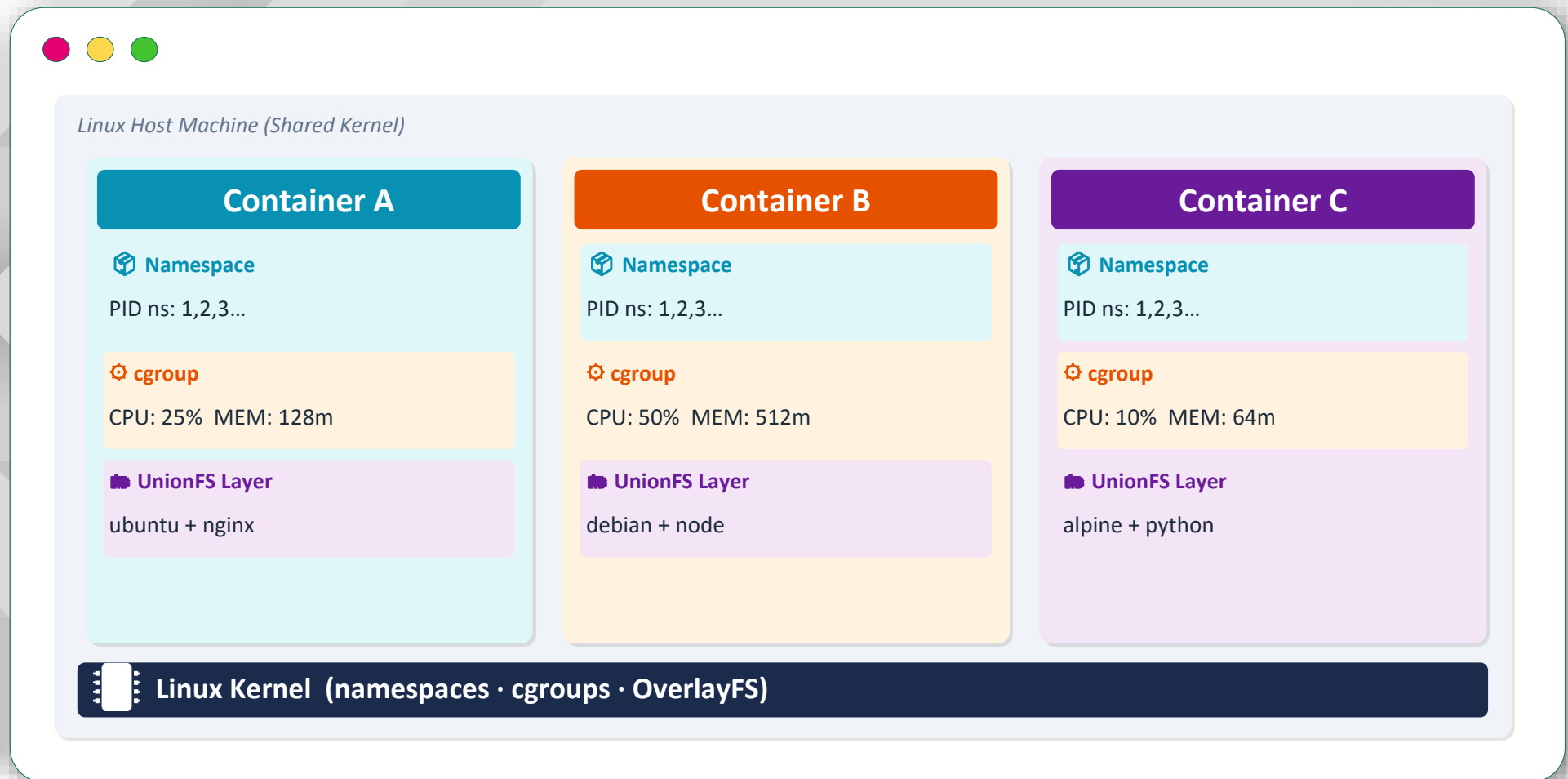
Each Dockerfile instruction (RUN, COPY, ADD) creates a new immutable layer — exactly like a commit.



Copy-on-Write (CoW)

When a container modifies a file from a lower layer, it copies it up into its own R/W layer. Originals stay intact.

How the Three Primitives Work Together



What Containers Give Us



Speed

Start in milliseconds, not minutes. No OS boot needed.



Isolation

Processes, networks, and filesystems are separate per container.



Efficiency

Share the kernel — run 10x more workloads on same hardware.



Consistency

Same container runs identically on any machine with Docker.



Portability

Build once, run anywhere — laptop, CI server, cloud.



Reproducibility

Dockerfile = exact recipe, version-controlled, shareable.



Part II

Docker

Images, Containers, Dockerfile & the CLI

What is Docker?



Docker is an open-source platform that packages your application and all its dependencies into a standardized unit called a container.

Docker Engine

The daemon (dockerd) that builds & runs containers — the heart of Docker

Docker CLI

Command-line tool to communicate with the Docker daemon via REST API

Docker Hub

Cloud registry storing 100,000+ pre-built images (like GitHub for images)

Dockerfile

Text recipe describing how to build a Docker image, step by step

Core Docker Objects



Image

Read-only template with instructions for creating a container. Layers stacked on each other. Like a class definition.



Container

A runnable instance of an image. Like an object instantiated from a class. Isolated process.



Dockerfile

Text file with instructions to build an image. Your recipe.



Registry

Storage & distribution system for Docker images. Docker Hub is the default public registry.



Volume

Persistent storage that lives outside container lifecycle. Data survives container restarts.



Network

Virtual network enabling containers to communicate. Isolation and connectivity.

Core Concepts: Image vs Container



Docker IMAGE

- Read-only blueprint/template
- Built from a Dockerfile
- Stored as layers on disk
- Shareable via registries
- Like a class definition in OOP
- Example: ubuntu:22.04, python:3.11



docker run

CONTAINER

- Running instance of an image
- Has its own writable layer
- Ephemeral by default
- Can be started/stopped/deleted
- Like an object instance in OOP
- Many containers from one image

Docker Image Layers



Ubuntu 22.04 (Base)

Python 3.11

pip install requirements

Your App Code

Why Layers Matter:

- Each instruction in Dockerfile = one layer
- Layers are cached — only changed layers rebuild
- Layers are shared between images (saves disk)
- Union filesystem merges all layers at runtime
- Read-only layers + thin R/W layer on top
- Pull only missing layers (bandwidth savings)



Order your Dockerfile instructions: change-rarely layers first, change-often layers last!

Image Layers & Caching



Layer 6: CMD ["uvicorn", "main:app"]

App
command

Layer 5: COPY . .

App source

Layer 4: RUN pip install -r requirements.txt

Dependen
cies

Layer 3: COPY requirements.txt .

Dep file

Layer 2: WORKDIR /app

Dir setup

Layer 1: FROM python:3.11-slim

Base
image

↑ Built top-to-bottom, cached bottom-to-top



Cache Invalidation

When a layer changes, ALL layers above it are rebuilt.



Best Practice:

Copy requirements.txt BEFORE copying source code.

If only your code changes:

- pip install is CACHED ✓
- Build is much faster ✓



Anti-pattern:

COPY . .

RUN pip install ← always re-runs!

Writing a Dockerfile

```

# 1. Start from an official base image
FROM python:3.11-slim

# 2. Set working directory inside container
WORKDIR /app

# 3. Copy dependency file first (layer caching!)
COPY requirements.txt .

# 4. Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy application code
COPY . .

# 6. Expose the port
EXPOSE 8000

# 7. Set environment variable
ENV PYTHONUNBUFFERED=1

# 8. Default command to run
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

FROM

Base image — starts your layer stack

WORKDIR

Sets working dir; creates it if missing

COPY

Copies files from host into image

RUN

Executes a shell command during build

EXPOSE

Documents which port the app uses

CMD

Default command when container starts

Build & Run Containers

Common run flags:

<code>-d</code>	Detached mode (runs in background)
<code>-p 8080:8000</code>	Map host port 8080 → container port 8000
<code>--name</code>	Give the container a friendly name
<code>-e KEY=VAL</code>	Set environment variables
<code>-v /host:/app</code>	Mount a volume (persistent storage)
<code>--rm</code>	Auto-remove container when it exits

```
# Build an image from Dockerfile in current directory
$ docker build -t myapp:1.0 .

# View local images
$ docker images

# Run a container
$ docker run -d -p 8080:8000 --name my-web myapp:1.0
#           ^      ^           ^
#           |      |           | name the container
#           |      |           | host:container port mapping
#           |      |           | detached (background) mode

# Check running containers
$ docker ps

# View logs
$ docker logs my-web

# Stop and remove
$ docker stop my-web && docker rm my-web
```

Volumes



```
# Named volume (managed by Docker)
docker run -v mydata:/var/lib/postgresql/data postgres

# Bind mount (maps host directory)
docker run -v $(pwd)/app:/app myimage

# Read-only mount
docker run -v /host/config:/app/config:ro myimage

# List volumes
docker volume ls
```

Volume types:

Named Volume:

Managed by Docker, survives container removal

Bind Mount:

Maps host path — good for development

tmpfs Mount:

In-memory only — useful for secrets

Networking

```
# Create a custom network
docker network create mynet

# Run containers on same network
docker run -d --network mynet --name web myapp
docker run -d --network mynet --name db postgres

# Containers can reach each other by NAME
# Inside 'web' container:
curl http://db:5432

# Network types
docker network ls
```

Network drivers:

bridge

Default — containers on same host communicate

host

Shares host's network stack (Linux only)

none

No networking — fully isolated

overlay

Multi-host — used by Docker Swarm



Part III

Docker Compose

Defining & Running Multi-Container Applications

Why Docker Compose?



Running a typical web app manually:



```
$ docker network create app-net
$ docker volume create pg-data
$ docker run -d --name db --network app-net -v pg-data:/var/lib/postgresql/data \
  -e POSTGRES_DB=mydb -e POSTGRES_PASSWORD=secret postgres:15
$ docker run -d --name redis --network app-net redis:7
$ docker run -d --name web --network app-net -p 8000:8000 \
  -e DATABASE_URL=postgresql://postgres:secret@db:5432/mydb \
  -e REDIS_URL=redis://redis:6379 myapp:1.0
$ docker run -d --name worker --network app-net \
  -e DATABASE_URL=postgresql://postgres:secret@db:5432/mydb \
  -e REDIS_URL=redis://redis:6379 myapp:1.0 celery -A tasks worker
```

✓ With Docker Compose:

1. Define your entire stack in a single docker-compose.yml file
2. run it with one command
3. commit to Git
4. everyone on the team has the same environment.

Anatomy of docker-compose.yml



```
services:
  web:
    build: .                # Build from local Dockerfile
    ports:
      - "8000:8000"        # host:container
    environment:
      - DATABASE_URL=postgresql://postgres:secret@db/mydb
      - REDIS_URL=redis://redis:6379
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    volumes:
      - ./app              # Bind mount for dev hot-reload
```

services:

Each entry is a container

build: .

Build from Dockerfile in current dir

ports:

Port mapping host → container

environment:

Env vars passed to container

depends_on:

Start order & health conditions

volumes:

Persistent data storage

healthcheck:

Docker polls this to verify readiness

Essential Compose Commands



```
$ docker compose up
```

Start all services (foreground)

```
$ docker compose up -d
```

Start all services in background (detached)

```
$ docker compose up --build
```

Rebuild images before starting

```
$ docker compose down
```

Stop and remove containers & networks

```
$ docker compose down -v
```

Also remove volumes (⚠ deletes data!)

```
$ docker compose ps
```

List running services

```
$ docker compose logs -f web
```

Stream logs from 'web' service

```
$ docker compose exec web bash
```

Open shell inside running container

```
$ docker compose run web pytest
```

Run one-off command in a new container

```
$ docker compose stop
```

Stop without removing containers

```
$ docker compose restart web
```

Restart a specific service

```
$ docker compose build
```

Build or rebuild images only

Real-World Example: FastAPI + PostgreSQL + Redis

```
# main.py
from fastapi import FastAPI
import redis, asyncpg, os

app = FastAPI()
r = redis.from_url(os.environ["REDIS_URL"])

@app.get("/items/{item_id}")
async def get_item(item_id: int):
    # Try cache first
    cached = r.get(f"item:{item_id}")
    if cached:
        return {"source": "cache", "data": cached}
    # Fetch from DB
    conn = await asyncpg.connect(os.environ["DATABASE_URL"])
    row = await conn.fetchrow(
        "SELECT * FROM items WHERE id=$1", item_id)
    r.setex(f"item:{item_id}", 60, str(row))
    return {"source": "db", "data": dict(row)}
```

Services:

web

image: myapp:1.0
port: 8000:8000

db

image: postgres:15
port: 5432 (internal)

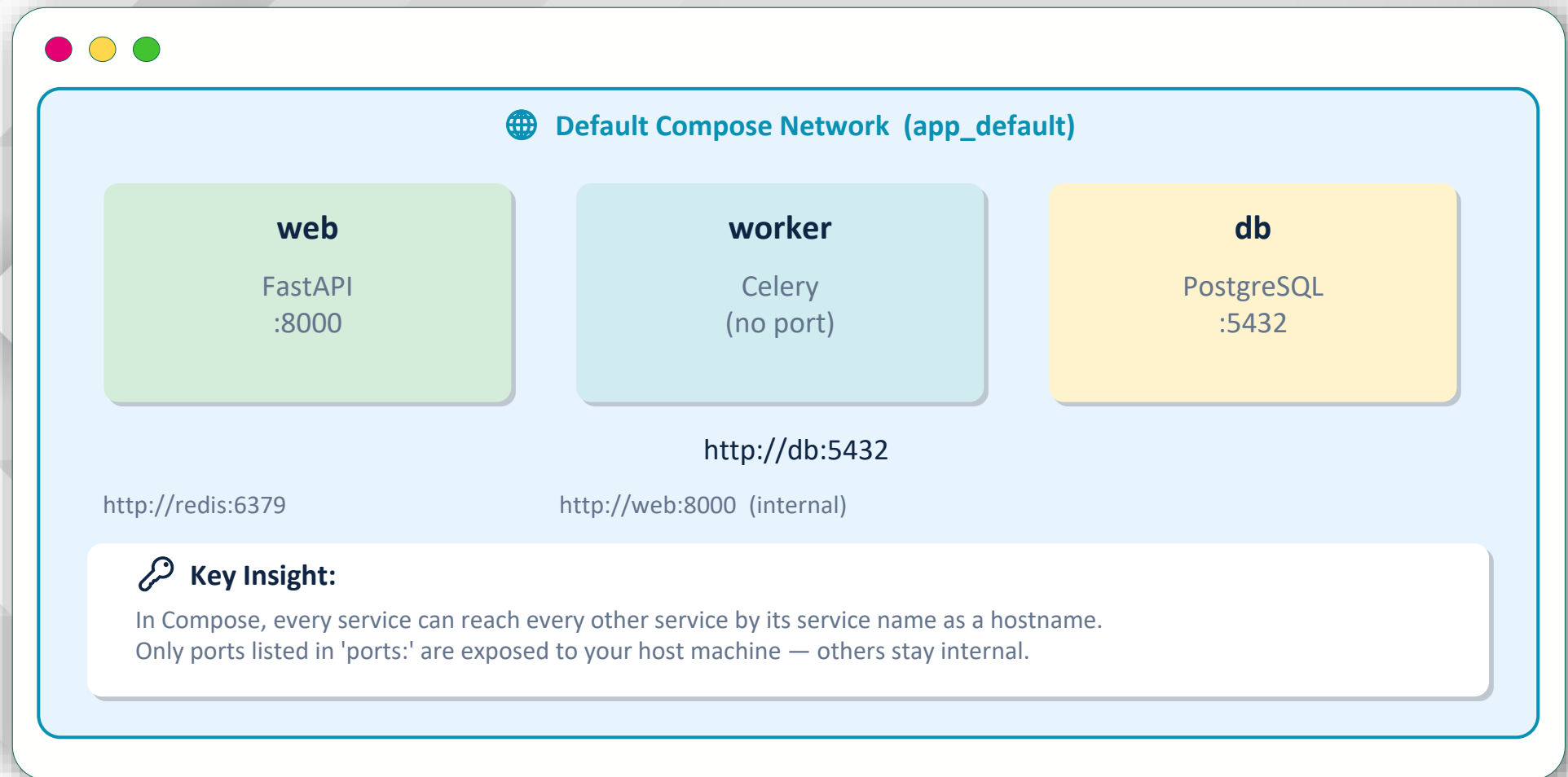
redis

image: redis:7-alpine
port: 6379 (internal)

```
$ docker compose up --build
```

→ All 3 services running!

Service Discovery in Compose



Compose Overrides & Profiles

Typical Project

```
docker-compose.yml
├── docker-compose.override.yml
│   (dev)
├── docker-compose.prod.yml
│   (production)
├── docker-compose.test.yml
│   (CI/CD)
└── .env
    (secrets, not in Git!)
```

Development

```
$ docker compose -f docker-compose.yml \
  -f docker-compose.override.yml up -d
```

Production

```
$ docker compose -f docker-compose.yml \
  -f docker-compose.prod.yml up -d
```

Profiles (enable optional services)

```
$ docker compose --profile debug up
```

```
# docker-compose.override.yml (auto-applied in dev)
```

```
services:
```

```
  web:
```

```
    volumes:
```

```
      - ./app # hot reload in dev
```

```
    command: uvicorn main:app --reload
```

```
    environment:
```

```
      - DEBUG=true
```

```
# docker-compose.prod.yml (applied explicitly)
```

```
services:
```

```
  web:
```

```
    deploy:
```

```
      replicas: 3
```

```
    environment:
```

```
      - DEBUG=false
```



Part IV

Docker Swarm

Container Orchestration & Clustering

The Problem: One Server is a Gamble

Without Swarm

Server (1 machine)

 web db worker

Server crashes at 3am

Everything goes down.
No web. No database. No workers.
You get a 3am phone call. 

 Downtime = lost money + angry users

With Swarm

 Server A web Server B web Server C 

Server C crashes at 3am

Swarm detects it in seconds.

→ Moves its containers to A & B automatically.
Users notice nothing. You sleep. 

 Zero downtime. Self-healing.

What Is a Docker Swarm?



A Swarm is a group of machines (servers) running Docker, all working together as one cluster.

 Think of it like a restaurant kitchen:



Head Chef

Decides who cooks what.
Reads the orders. Plans everything.

=

Manager Node (Leader)

Schedules containers.
Makes all cluster decisions.



Sous Chef

Watches the head chef.
Ready to take over if needed.

=

Manager Node (Replica)

Standby manager.
Becomes leader if current leader fails.



Line Cooks

Just cook. Don't plan.
Do exactly what they're told.

=

Worker Nodes

Just run containers.
No scheduling decisions.



A dish order

"Make 3 portions of pasta"

=

A Task (container replica)

"Run 3 copies of the web service"

Managers vs. Workers



MANAGERS



Keep the desired state

"I want 3 web containers running at all times"



Schedule tasks

Decide which worker runs which container



Monitor everything

Constantly check if all tasks are healthy



Vote on decisions

Use Raft to agree before making changes



Expose the API

You talk to the manager — it handles the rest



WORKERS



Run containers

That's it. Receive task → start container



Report health status

Tell manager: "my containers are healthy"



No scheduling

Workers never decide where containers run



No voting

Workers don't participate in Raft elections



Easy to scale

Add more workers = more capacity instantly

Raft: How Managers Agree on Decisions



Problem: if 3 managers can all make decisions independently, they'd contradict each other and corrupt the cluster.
Solution: only **ONE manager makes** decisions (the Leader) — and it's chosen by a vote.

1

Normal operation — Leader sends heartbeats

The Leader sends a "I'm alive" signal to all other managers every few hundred milliseconds.

M1 (Leader) —♥→ M2
—♥→ M3

2

Leader goes silent — followers notice

M1 crashes. M2 and M3 stop receiving heartbeats. After a timeout (~150ms) they suspect the leader is dead.

M1 ✕ (no heartbeat)
M2: "I haven't heard from leader..."
M3: "Me neither..."

3

Election — any follower nominates itself

M2 nominates itself. It sends a vote request to M3. M3 says YES. M2 now has 2 votes out of 3 — that's a majority.

M2: "Vote for me!" → M3
M3: "Yes, I vote M2" → M2
M2 has 2/3 votes → NEW LEADER ✓

4

New Leader takes over — cluster continues

M2 is now Leader. It resumes scheduling and heartbeats. The whole election took milliseconds. Users noticed nothing.

M2 (Leader) —♥→ M3
Cluster fully operational ✓

Quorum: Why You Need Odd Numbers of Managers



Quorum = the minimum number of managers that must be alive to make decisions.

$$\text{Formula: Quorum} = \lfloor \text{Total Managers} / 2 \rfloor + 1$$

Managers	Quorum needed	Can lose	Alive needed	Verdict
	1	0 failures	1 of 1	⚠️ OK for dev only
	2	0 failures	2 of 2	✗ WORSE than 1!
	2	1 failure	2 of 3	☑ Minimum for prod
	3	1 failure	3 of 4	⚠️ Same as 3, wasteful
	3	2 failures	3 of 5	☑ Recommended

How a Service Gets Deployed



```
$ docker service create --name web --replicas 3 -p 80:8000 myapp:1.0
```

1**You run the command**

Sent to the Manager (Leader) via REST API

2**Manager stores the desired state**

"I want 3 copies of web running at all times"

3**Scheduler finds best workers**

Checks CPU/RAM available on each node, spreads replicas across different workers

4**Manager assigns tasks to workers**

Task 1 → Worker A, Task 2 → Worker B, Task 3 → Worker C

5**Workers pull image & start containers**

Each worker downloads the image and starts its assigned container

6**Manager watches forever**

If any container dies — immediately schedules a replacement on another worker

Swarm Command Cheat Sheet



Setup

```
$ docker swarm init
```

Turn this machine into a Swarm manager (Leader)

```
$ docker swarm join --token ... ip
```

Add a worker to the Swarm

```
$ docker node ls
```

List all nodes and their roles

Scaling & Updates

```
$ docker service scale web=5
```

Scale web service to 5 replicas

```
$ docker service update --image myapp:2.0 web
```

Update to a new image (rolling)

```
$ docker service rollback web
```

Undo the last update instantly

Services

```
$ docker service create --name web --replicas 3 myapp:1.0
```

Deploy a service with 3 replicas

```
$ docker service ls
```

List all running services

```
$ docker service ps web
```

Show which node each replica is on

Stacks (Compose for Swarm)

```
$ docker stack deploy -c docker-stack.yml myapp
```

Deploy entire stack from YAML file

```
$ docker stack services myapp
```

List services in a stack

```
$ docker stack rm myapp
```

Remove entire stack

Key Takeaways



1 Docker Basics

- Containers package app + environment together
- Dockerfile → Image → Container lifecycle
- Layer caching keeps builds fast
- Volumes persist data, networks connect containers

2 Docker Compose

- Define multi-container apps in one YAML file
- Services communicate using their service name as hostname
- `docker compose up -d` is your daily dev command
- Override files for dev vs production configs

3 Docker Swarm

- Orchestration adds scale, self-healing, rolling updates
- Managers schedule tasks; Workers run them
- Stack deploy = Compose + Swarm deploy key
- Swarm is simpler than K8s; K8s is industry standard

Further Learning & Resources



Official Docs

- docs.docker.com — full reference
- docs.docker.com/compose — Compose spec
- docs.docker.com/guides

Tools to Explore

- Portainer — GUI for Docker management
- Lazydocker — terminal UI for containers
- Dive — analyze image layers & size
- Trivy — container vulnerability scanning



Thank You!

Questions?